PROJECT REPORT

GPU-Accelerated Deep Learning-Based ECG Signal Classification for Cardiac Abnormality Detection

Using the MIT-BIH Arrhythmia Database, TensorFlow/Keras, 1D-CNN, CNN-LSTM, and CPU–GPU Benchmarking

Submission Detail	Entry
Project / Group	Group 7 — Deep Learning Project
Team Member 1	Ronit Jana — 24/CSE/145
Team Member 2	Ritam Jana — 24/CSE/138
Team Member 3	Rishi Raj — 24/CSE/137
Team Member 4	Rohan Adak — 24/CSE/143
Team Member 5	Rishabh Jha — 24/CSE/136
Department	Computer Science & Engineering
Course / Subject	_____
Faculty	_____
Date of Submission	_____

All numerical results in this report are taken from the experiment files recorded in the project repository (results/*.csv and results/*.json).

Abstract

This report presents the design, implementation and evaluation of a GPU-accelerated deep-learning prototype that classifies single ECG heartbeats as Normal or Abnormal. The system uses the MIT-BIH Arrhythmia Database from PhysioNet, of which 44 usable records are filtered, segmented into 360-sample beat windows and split record-wise (patient-wise) into training, validation and test sets. Two TensorFlow/Keras models are trained on the same data: a 1D Convolutional Neural Network (1D-CNN) and a CNN-LSTM. On a test set of 15,897 beats from 7 completely unseen records, the 1D-CNN achieved 80.66% accuracy, 0.5166 F1-score and 0.8448 ROC-AUC for the abnormal class, while the CNN-LSTM achieved 76.05% accuracy, 0.4479 F1-score and 0.7418 ROC-AUC. To study GPU acceleration, the CNN-LSTM was trained and evaluated on both CPU and a Tesla T4 GPU with identical settings. The GPU reduced total five-epoch training time from 297.20 s to 33.94 s (8.76× speed-up) and inference time per beat from 0.2241 ms to 0.0302 ms (7.42× speed-up). The models are integrated in an interactive Streamlit demo. This prototype is strictly an academic demonstration and must not be interpreted as a clinical diagnostic system.

1. Introduction

An electrocardiogram (ECG) records the electrical activity of the heart. Every heartbeat produces a characteristic waveform, and abnormal beats such as premature atrial or ventricular contractions change the shape and timing of this waveform. Inspecting long ECG recordings manually is slow and tiring, so automatic beat classification is a natural application of machine learning in smart healthcare.

Deep learning is well suited to this task because a convolutional network can learn waveform features directly from raw samples without hand-crafted features, and a recurrent layer such as an LSTM can additionally model how those features evolve over time. These networks consist largely of matrix and tensor operations that can be executed in parallel, which makes Graphics Processing Units (GPUs) very effective for accelerating them.

The objective of this project is not to build a clinically deployable arrhythmia detector. Instead, the goal is to build a complete and leakage-free ECG classification pipeline, evaluate it honestly on unseen patients, and experimentally measure how GPU acceleration changes training and inference time compared with a CPU. The project connects four ideas: ECG data handling, deep-learning-based classification, model evaluation, and CPU–GPU performance comparison.

2. Objectives

- Load, analyse and preprocess the MIT-BIH ECG recordings (filtering, R-peak-centred segmentation, normalisation).
- Split the data record-wise so that no patient appears in more than one of train, validation and test sets.
- Implement and train a 1D-CNN and a CNN-LSTM in TensorFlow/Keras for Normal versus Abnormal beat classification.
- Handle the strong class imbalance using balanced class weights and evaluate with recall, F1-score, specificity, ROC-AUC and confusion matrices, not accuracy alone.

- Run the same CNN-LSTM training and inference workload on CPU and on a CUDA-enabled GPU.
- Compute the GPU speed-up ratios and interpret the results.
- Build an interactive Streamlit demo that connects the whole pipeline.
- Understand the limitations and non-clinical nature of an academic healthcare AI prototype.

3. Tools and Technologies

Component	Used in this work
Programming Language	Python 3.13.15
Deep Learning Framework	TensorFlow 2.20.0 / Keras 3.13.2
ECG Data Library	WFDB (wfdb-python)
Signal Processing	SciPy (Butterworth band-pass filter), NumPy
Dataset	MIT-BIH Arrhythmia Database (PhysioNet)
GPU Runtime	NVIDIA Tesla T4, CUDA 12.5.1, cuDNN 9, driver 580.82.07
Notebook Environment	Google Colab
Metrics	scikit-learn
Visualization	Matplotlib
Demo Application	Streamlit

4. Dataset Description

The MIT-BIH Arrhythmia Database from PhysioNet contains 48 half-hour ECG recordings sampled at 360 Hz. Each record has three files: a .dat file (signal), a .hea file (header) and a .atr file with beat annotations placed at each R-peak. The records were downloaded with the WFDB Python package, and the MLII lead (channel 0) was used. Four paced records (102, 104, 107 and 217) were excluded, so 44 of the 48 records are used.

Item	Value
Source	MIT-BIH Arrhythmia Database (PhysioNet)
Sampling rate	360 Hz
Lead used	MLII (channel 0)
Records downloaded / used	48 / 44
Total beats	100,733
Normal beats	90,125
Abnormal beats	10,608 (10.5%)

The team defined the binary labels from the beat annotation symbols as follows:

Class	Annotation symbols
Normal (0)	N, L, R, e, j
Abnormal (1)	A, a, J, S, V, E, F, /, f, Q

Non-beat annotation symbols (for example "+") are ignored. Note that bundle-branch-block beats (L and R) are placed in the Normal class by this definition. The dataset is strongly imbalanced: only 10.5% of beats are abnormal. This affects both how the models are trained (Section 6) and how they must be judged (Section 7).

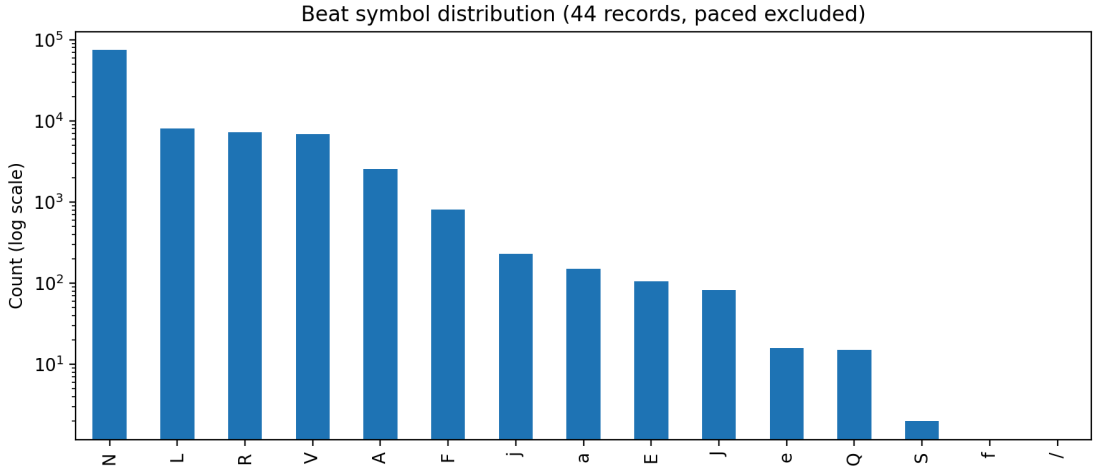


Figure 1. Beat-symbol counts in the dataset (log scale).

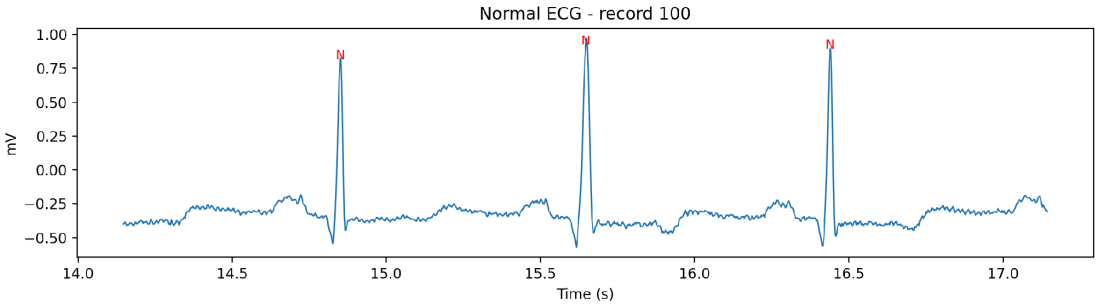


Figure 2. Example of a normal ECG segment (record 100).

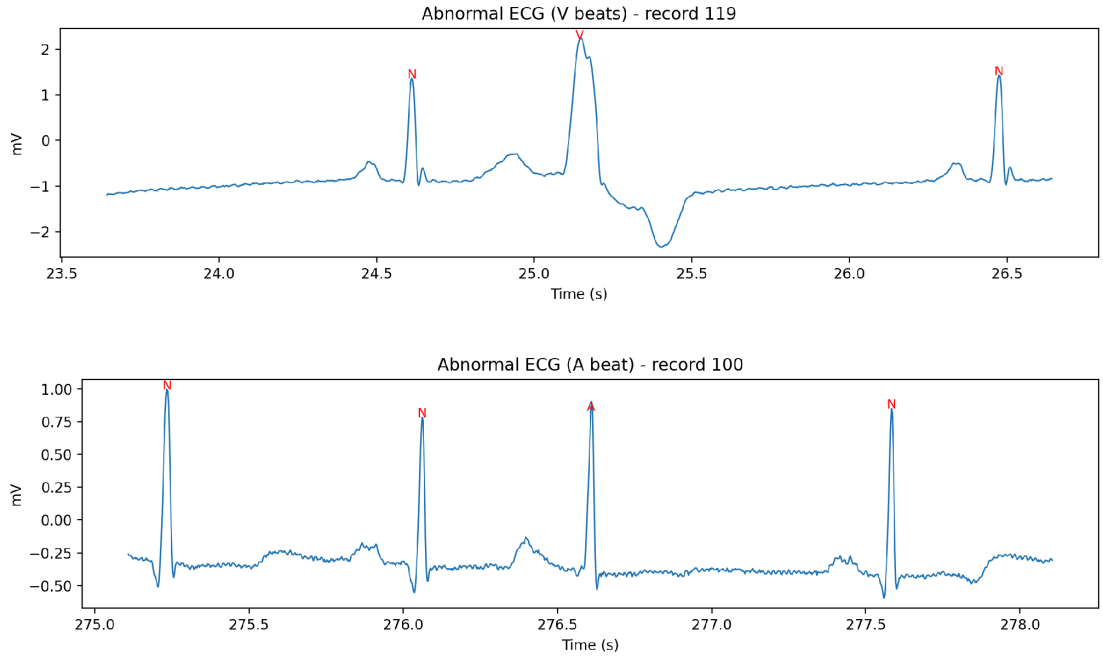


Figure 3. Examples of abnormal beats: ventricular beat (record 119, top) and atrial beat (record 100, bottom).

5. Methodology

5.1 Overall Workflow

1. Download the 44 MIT-BIH records with WFDB and read the MLII lead and the beat annotations.
2. Apply a 0.5–45 Hz Butterworth band-pass filter to each full record.
3. Cut a 360-sample window centred on each annotated R-peak and z-score normalise it.
4. Assign binary labels (Normal / Abnormal) from the annotation symbols.
5. Split the records into train, validation and test sets record-wise to prevent leakage.
6. Train the 1D-CNN and the CNN-LSTM with balanced class weights.
7. Evaluate both models once on the unseen test set.
8. Train and run inference of the same CNN-LSTM on CPU and GPU and compare the timings.
9. Integrate everything into the Streamlit demo.

5.2 Preprocessing and Data Split

The raw records were converted into fixed-size model inputs in four steps. First, a 4th-order Butterworth band-pass filter (0.5–45 Hz) removes baseline wander and high-frequency noise; it is applied to the whole record before segmentation. Second, a window of 180 samples before and 180 samples after each R-peak (360 samples, exactly 1 second) is extracted. Beats whose window does not fit inside the record are dropped, so 100,733 annotated beats become 100,682 segments. Third, each segment is z-score normalised as (segment − mean) / std. Fourth, the label is assigned from the annotation symbol.

```
def butter_bandpass_filter(data, lowcut=0.5, highcut=45.0, fs=360.0, order=4):
```

```
    nyquist = 0.5 * fs
    b, a = butter(order, [lowcut / nyquist, highcut / nyquist], btype='band')
    return filtfilt(b, a, data)

# for every annotated beat at sample index idx:
segment = filtered_signal[idx - 180 : idx + 180]          # 360 samples
segment = (segment - np.mean(segment)) / np.std(segment)  # z-score
```

The split is **record-wise (patient-wise)**: every record appears in only one of the three sets. This avoids data leakage, because beats from the same person look very similar and would otherwise inflate the test results.

Split	Records	Beats	Normal	Abnormal	Abnormal %
Train	30	68,386	62,790	5,596	8.2%
Validation	7	16,399	14,553	1,846	11.3%
Test	7	15,897	12,733	3,164	19.9%

The test records are 207, 223, 230, 231, 232, 233 and 234; the validation records are 203, 214, 219, 220, 221, 222 and 228. The processed data was saved as `ecg_processed_v1.npz` (about 134 MB) with float32 arrays `X_train` (68386, 360, 1), `X_val` (16399, 360, 1) and `X_test` (15897, 360, 1) plus matching int32 label arrays. Both models use this same file. Note that the abnormal share is 8.2% in the training set but 19.9% in the test set, which is important when reading the results.

5.3 Model Architectures

Two models are compared. The 1D-CNN uses convolution filters that slide along the signal and learn local waveform patterns such as the shape of the QRS complex, and pooling layers reduce the length while keeping the strongest responses. The CNN-LSTM first uses three convolution blocks to extract local features, and then an LSTM layer to learn how these features change along the beat. Both models end with a single sigmoid unit whose output is the probability that the beat is abnormal; a threshold of 0.5 gives the class.

```
# 1D-CNN
layers.Input(shape=(360, 1)),
layers.Conv1D(32, kernel_size=5, padding="same", activation="relu"),
layers.MaxPooling1D(pool_size=2),
layers.Conv1D(64, kernel_size=5, padding="same", activation="relu"),
layers.MaxPooling1D(pool_size=2),
layers.Flatten(),
layers.Dense(64, activation="relu"),
layers.Dropout(0.5),
layers.Dense(1, activation="sigmoid")
```

Layer / Block	Output shape	Parameters	Purpose
Conv1D (32, k=5) + ReLU	360 × 32	192	Low-level waveform features
MaxPool1D (2)	180 × 32	0	Reduce length
Conv1D (64, k=5) + ReLU	180 × 64	10,304	Higher-level features
MaxPool1D (2)	90 × 64	0	Reduce length
Flatten	5,760	0	Vectorise
Dense (64) + ReLU	64	368,704	Compact representation
Dropout (0.5)	64	0	Reduce overfitting
Dense (1) + Sigmoid	1	65	P(abnormal)
Total		379,265	

```
# CNN-LSTM
inp = keras.Input(shape=(360, 1))
x = layers.Conv1D(32, 5, padding='same', activation='relu')(inp)
x = layers.MaxPooling1D(2)(x)                      # 360 -> 180
x = layers.Conv1D(64, 5, padding='same', activation='relu')(x)
x = layers.MaxPooling1D(2)(x)                      # 180 -> 90
x = layers.Conv1D(128, 3, padding='same', activation='relu')(x)
x = layers.MaxPooling1D(2)(x)                      # 90 -> 45 time-steps
x = layers.LSTM(64)(x)
x = layers.Dense(32, activation='relu')(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(1, activation='sigmoid')(x)
```

Layer / Block	Output shape	Parameters*	Purpose
Conv1D (32, k=5) + ReLU	360 × 32	192	Local waveform features
MaxPool1D (2)	180 × 32	0	Reduce length
Conv1D (64, k=5) + ReLU	180 × 64	10,304	Higher-level features
MaxPool1D (2)	90 × 64	0	Reduce length
Conv1D (128, k=3) + ReLU	90 × 128	24,704	Higher-level features
MaxPool1D (2)	45 × 128	0	45 time-steps for LSTM
LSTM (64)	64	49,408	Temporal dependencies
Dense (32) + ReLU	32	2,080	Compact representation
Dropout (0.3)	32	0	Reduce overfitting
Dense (1) + Sigmoid	1	33	P(abnormal)
Total		86,721	

*The CNN-LSTM parameter counts are calculated from the layer shapes; the 1D-CNN total of 379,265 matches the value recorded in `results/1d_cnn_metrics.csv`.

6. Model Training

Both models are trained with the Adam optimiser and binary cross-entropy loss. Because only 8.2% of the training beats are abnormal, balanced class weights computed from the training set are used (Normal 0.5446, Abnormal 6.1103), so that mistakes on abnormal beats are penalised more strongly. Early stopping on the validation loss restores the best weights. Training used only the train and validation sets; the test set was used once, for the final evaluation. A fixed random seed (42) is used for repeatability, although exact numerical results can still vary between platforms.

```
keras.utils.set_random_seed(SEED)                  # SEED = 42
with tf.device(device):                             # '/GPU:0' or '/CPU:0'
    model = build_cnn_lstm()
    cbs = [EpochTimer(),
           keras.callbacks.EarlyStopping(monitor='val_loss', patience=4,
                                         restore_best_weights=True)]
    hist = model.fit(X_train, y_train,
                     validation_data=(X_val, y_val),
                     epochs=epochs, batch_size=128,
                     class_weight=class_weight,    # {0: 0.5446, 1: 6.1103}
                     shuffle=True, callbacks=cbs)
```

Training Parameter	1D-CNN	CNN-LSTM
Seed / batch size	42 / 128	42 / 128
Optimizer	Adam, lr 0.001	Adam, lr 0.001
Loss function	Binary cross-entropy	Binary cross-entropy
Class weights	Normal 0.5446, Abnormal 6.1103	Normal 0.5446, Abnormal 6.1103
Maximum epochs	30	15
Early stopping (val_loss)	patience 6	patience 4
ReduceLROnPlateau	Yes (factor 0.5, patience 2, min 1e-5)	No
Epochs run / best epoch	7 / 1	13 / 9
Training device	CPU (no GPU visible)	GPU (Tesla T4)
Training time	462.6 s	112.11 s
Precision	float32	float32 (no mixed precision)

Important: the maximum epochs, early-stopping patience, learning-rate schedule and hardware differ between the two models. Therefore the two training times must not be compared directly; the controlled CPU-versus-GPU comparison is reported separately in Section 8.

7. Experimental Results

Note: All values in this section are the real measured outputs recorded in `results/*.csv`. Every experiment was run once.

7.1 Training and Validation Performance

The 1D-CNN reached its best validation loss (0.4543) and validation accuracy (0.8145) already at epoch 1. Early stopping ended training after 7 epochs and restored the epoch-1 weights. After epoch 1 the training accuracy rose to about 98.8% while validation loss increased, which shows overfitting to the training records.

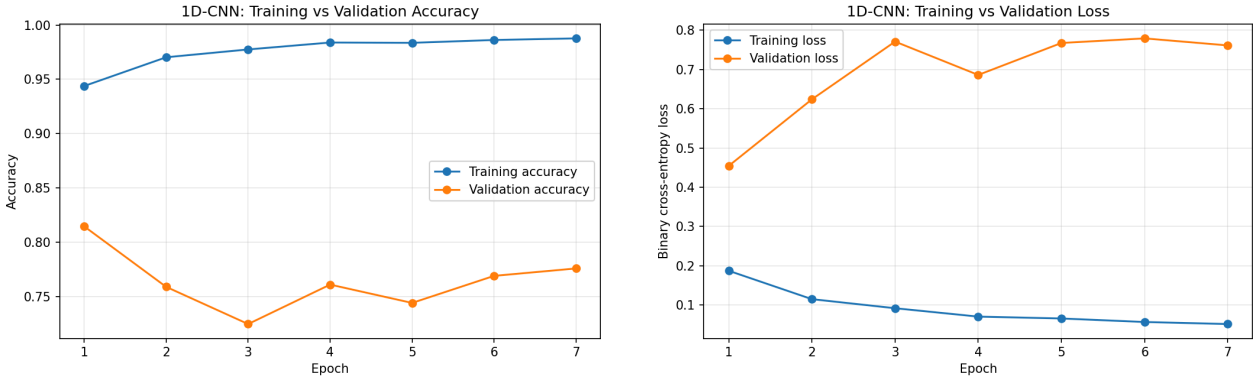


Figure 4. 1D-CNN training vs validation accuracy (left) and loss (right).

The CNN-LSTM training history is summarised below (selected epochs). The model ran 13 epochs and the best validation loss (0.5741) occurred at epoch 9.

Epoch	Training Loss	Validation Loss	Training Acc.	Validation Acc.
1	0.2553	0.6919	92.53%	70.09%
3	0.1134	0.7424	96.86%	75.73%
5	0.0807	0.6494	97.64%	78.25%
7	0.0663	0.6015	98.03%	84.73%
9	0.0548	0.5741	98.28%	82.26%
11	0.0480	0.6021	98.42%	82.32%
13	0.0510	1.0250	98.39%	75.02%

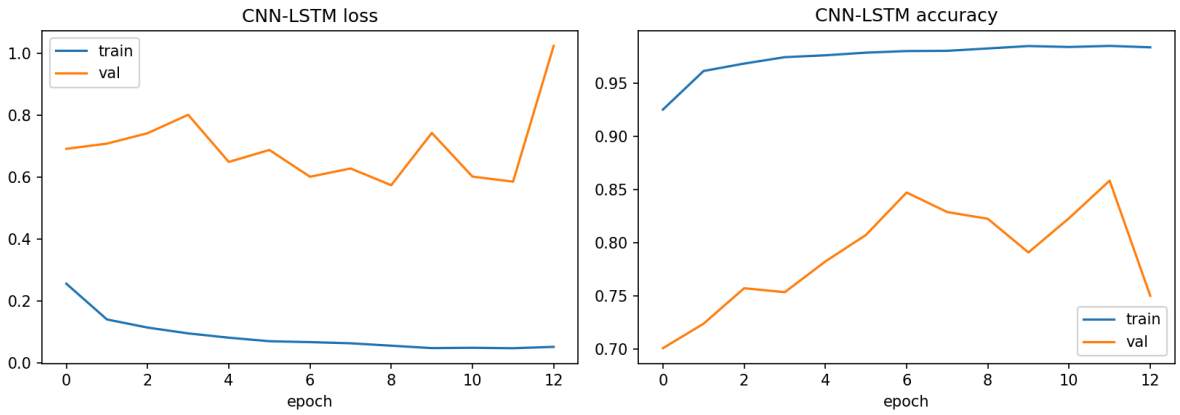


Figure 5. CNN-LSTM training and validation loss/accuracy.

Training loss falls steadily and training accuracy reaches about 98%, but the validation loss is unstable and does not improve in a clean downward trend. The models therefore fit the training records much better than new, unseen records. The CNN-LSTM also shows a clear gap between validation and test behaviour:

Metric (CNN-LSTM)	Validation set	Test set	Change
Accuracy	0.8226	0.7605	−0.0621
Precision (abnormal)	0.3733	0.4138	+0.0405
Recall (abnormal)	0.8483	0.4880	−0.3603
F1-score (abnormal)	0.5185	0.4479	−0.0706
Specificity	0.8193	0.8282	+0.0089
ROC-AUC	0.9171	0.7418	−0.1753

Recall for abnormal beats dropped from 0.85 on validation records to 0.49 on test records. The test set contains 19.9% abnormal beats versus 8.2% in training, and all test records are unseen patients, so this is a realistic indicator of how hard patient-independent generalisation is.

7.2 Test-Set Classification Metrics

The test set consists of 15,897 beats from 7 unseen records (12,733 Normal and 3,164 Abnormal), evaluated at threshold 0.5.

Metric	1D-CNN	CNN-LSTM
Accuracy	0.8066	0.7605
Precision (abnormal)	0.5139	0.4138
Recall / Sensitivity (abnormal)	0.5193	0.4880
F1-score (abnormal)	0.5166	0.4479
Specificity	0.8780	0.8282
ROC-AUC	0.8448	0.7418

Accuracy is the fraction of correctly classified beats. Precision describes how often an Abnormal prediction is correct, recall (sensitivity) shows how many truly abnormal beats are found, specificity shows how many normal beats are correctly kept as Normal, and the F1-score balances precision and recall. ROC-AUC measures how well the predicted probability separates the two classes across all thresholds. Because abnormal beats are the minority class, accuracy alone is misleading; for example, always predicting Normal would already give 80.1% accuracy on this test set (12,733 / 15,897) while detecting no abnormal beat at all.

For the 1D-CNN, the per-class report is:

Class	Precision	Recall	F1-score	Support
Normal (0)	0.8802	0.8780	0.8791	12,733
Abnormal (1)	0.5139	0.5193	0.5166	3,164
Macro average	0.6971	0.6986	0.6978	15,897
Weighted average	0.8073	0.8066	0.8069	15,897

7.3 Confusion Matrices

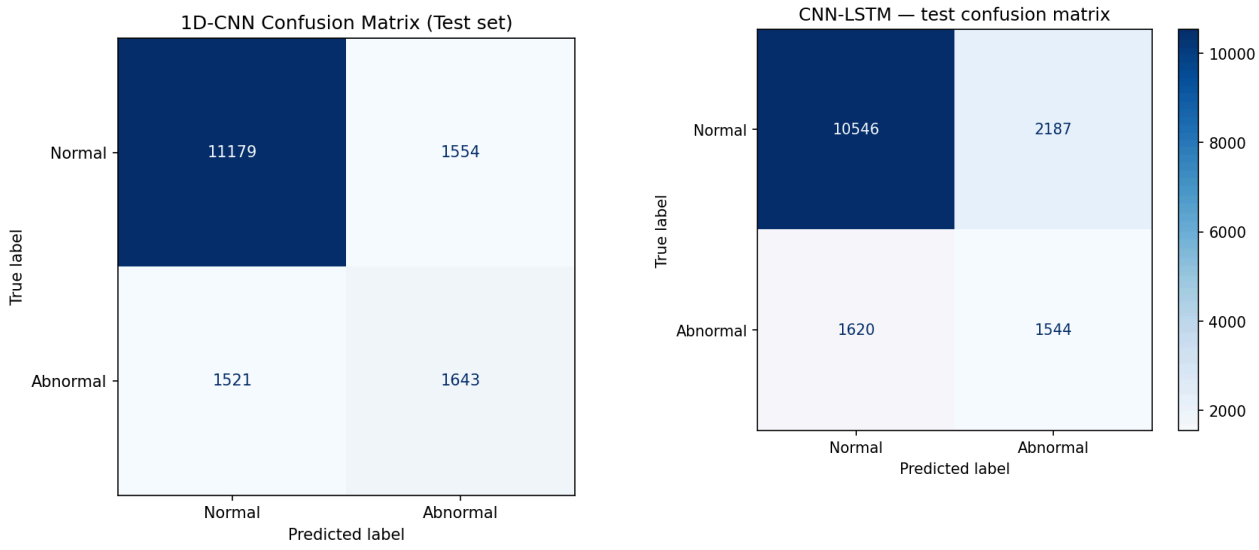


Figure 6. Test-set confusion matrices: 1D-CNN (left) and CNN-LSTM (right).

Model	TN	FP	FN	TP
1D-CNN	11,179	1,554	1,521	1,643
CNN-LSTM	10,546	2,187	1,620	1,544

Both models classify most normal beats correctly, but both miss a large share of abnormal beats: the 1D-CNN produces 1,521 false negatives and the CNN-LSTM 1,620 false negatives out of 3,164 abnormal beats. In a real clinical setting false negatives would be particularly important because a missed abnormal beat can have serious consequences; however, this classroom model has not undergone the validation required for clinical use. The CNN-LSTM also produces more false positives (2,187 versus 1,554), which explains its lower precision.

7.4 Model Comparison

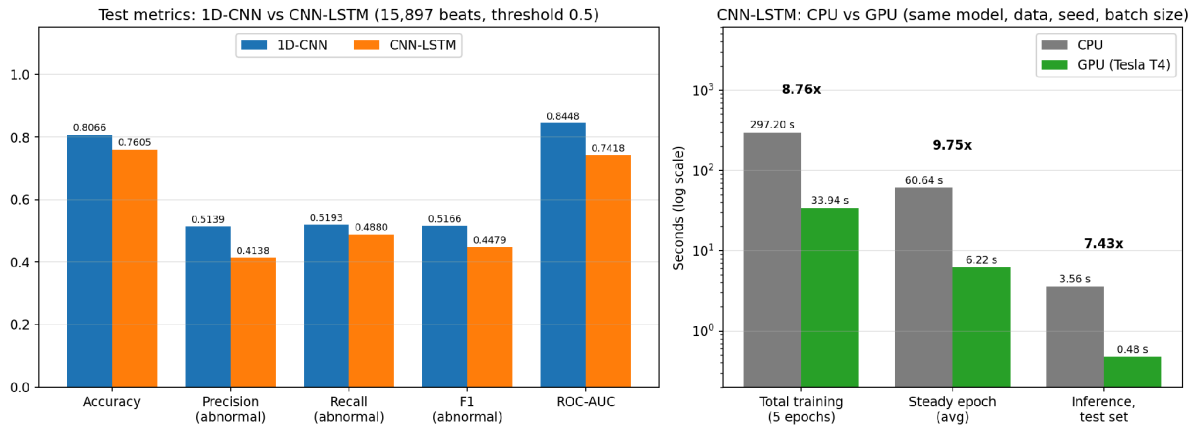


Figure 7. Final comparison of the 1D-CNN and the CNN-LSTM on the test set.

The simpler 1D-CNN is better than the CNN-LSTM on every metric in the comparison, so the extra LSTM layer did not help on this dataset. Another important finding is that the test result depends heavily on a few patients: three test records (232, 233 and 223) contain 2,791 of the 3,164 abnormal test beats, so the overall score is strongly influenced by how the model behaves on these records.

7.5 Interactive Streamlit Demo

A four-step Streamlit application connects the full pipeline: (1) select a built-in test beat (a sample of 400 real test beats is included) or upload an ECG file (.csv, .txt or .npy); (2) view the preprocessing; (3) obtain the predictions of both models together with P(abnormal); and (4) view the evaluation tables, which are read directly from the `results/*.csv` files. The application is available online at ecg-project-deeplearning-6383kyappwquwvbrmgodag6.streamlit.app and a demo video is available at youtu.be/or2yRLIMrqw. The prediction confidence is the sigmoid probability of the model and should not be interpreted as a medically calibrated probability of disease.

8. CPU–GPU Performance Benchmark

To make the comparison fair, the same CNN-LSTM, the same data, the same random seed (42), the same batch size (128) and the same number of epochs (5, without early stopping) were used for both devices. The

GPU run places the model on `/GPU:0` and the CPU run on `/CPU:0`, both inside the same Google Colab runtime. Epoch time includes validation time. Inference is timed on the full test set of 15,897 beats after a warm-up pass that builds the graph and initialises the GPU.

```
with tf.device(device):                       # '/CPU:0' or '/GPU:0'
    model.predict(X[:256 * 2], batch_size=256, verbose=0)   # warm-up
    t0 = time.perf_counter()
    model.predict(X, batch_size=256, verbose=0)             # full test set
    elapsed = time.perf_counter() - t0
ms_per_beat = 1000 * elapsed / len(X)
```

Hardware / Software	Value	Details
GPU	NVIDIA Tesla T4 (15,360 MiB)	Driver 580.82.07
CUDA / cuDNN	CUDA 12.5.1	cuDNN 9
CPU	Intel Xeon @ 2.00 GHz	2 logical cores
RAM	About 12.7 GiB	
Software	Python 3.13.15	TensorFlow 2.20.0, Keras 3.13.2
Recorded on	2026-10-04	Google Colab

Metric	CPU	GPU	Speed-up (CPU / GPU)
Total training time (5 epochs)	297.20 s	33.94 s	8.76×
Average epoch time (epoch 2 onward)	60.64 s	6.22 s	9.75×
First epoch (includes warm-up)	54.47 s	8.80 s	—
Inference, full test set (15,897 beats)	3.563 s	0.479 s	7.43×
Inference per beat	0.2241 ms	0.0302 ms	7.42×
Test accuracy after 5 epochs	0.7632	0.7671	—

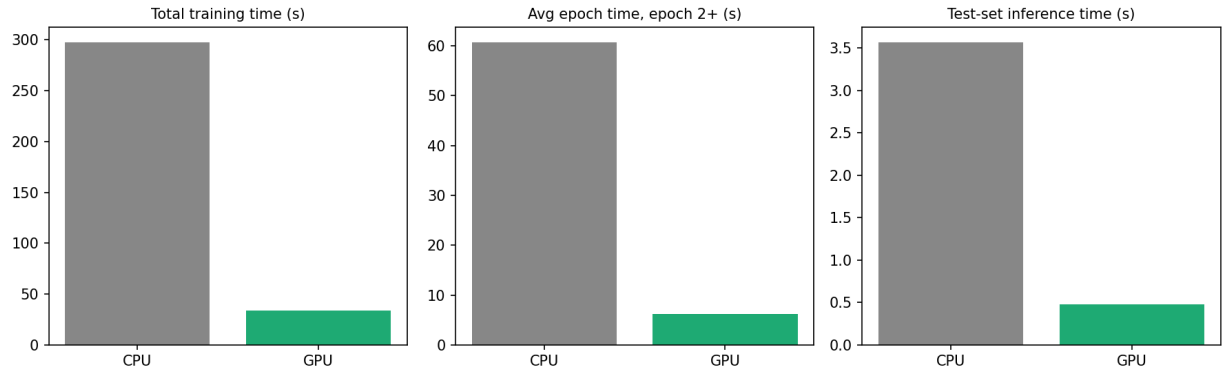


Figure 8. CPU vs GPU: training, epoch and inference time for the CNN-LSTM.

The speed-ups are calculated as:

Training speed-up = 297.20 / 33.94 = 8.76×

Steady-epoch speed-up = 60.64 / 6.22 = 9.75×

Inference speed-up = 3.563 / 0.479 = 7.43×

8.1 Interpretation

The GPU is faster in every measurement because the convolution, LSTM and dense operations of the network consist largely of tensor operations that can be executed in parallel on thousands of GPU cores, whereas the 2-core CPU can execute only a few at a time. The speed-up is about 9.75× for steady-state training epochs, 8.76× for total five-epoch training and 7.4× for inference. The total training speed-up is lower than the steady-epoch speed-up because the GPU's first epoch (8.80 s) is noticeably slower than its later epochs (6.22 s average), which indicates start-up and warm-up overhead; the CPU's first epoch (54.47 s) was not slower than its later epochs.

The CPU and GPU test accuracies differ slightly (0.7632 and 0.7671) because they are two separate five-epoch runs with different compute kernels; these values also differ from the main run (0.7605), which used early stopping. GPU execution improves computational speed; it does not automatically improve classification accuracy. The magnitude of the speed-up also depends on the GPU and CPU model, batch size, framework version and data-transfer overhead, so the numbers should only be compared with runs on the same hardware.

9. Key Observations

- The 1D-CNN outperforms the CNN-LSTM on every test metric (accuracy 0.8066 vs 0.7605, F1 0.5166 vs 0.4479, ROC-AUC 0.8448 vs 0.7418), so a more complex model is not automatically a better model.
- Both models are weak on the abnormal class when tested on unseen patients (F1 about 0.45 to 0.52, recall about 0.49 to 0.52).
- Both models overfit the training records: the 1D-CNN reached its best validation loss at epoch 1, and the CNN-LSTM reached about 98% training accuracy with unstable validation loss.
- The CNN-LSTM recall fell from 0.85 on validation records to 0.49 on test records, showing that patient-to-patient variation and a different class ratio strongly affect results.
- A record-wise split is essential. Splitting beats randomly would put similar beats from the same patient into both train and test sets and would give misleadingly high scores.
- The test score depends heavily on a few patients: records 232, 233 and 223 contain 2,791 of the 3,164 abnormal test beats.
- The GPU gave a 7.4× to 9.8× speed-up for the CNN-LSTM, depending on whether inference, total training or steady epoch time is measured.
- Medical AI evaluation requires more than accuracy alone; class imbalance, recall, specificity, ROC-AUC, calibration, robustness and clinical validation all matter.

10. Limitations

- **Overfitting:** both models fit the training records much better than unseen records.
- **Validation-versus-test gap:** the test set has a much higher abnormal share (19.9%) than the training set (8.2%), and all test records are unseen patients.
- **Single runs:** every experiment was run once (TIMING_REPEATS = 1, no repeated training), so small differences between models or devices should not be over-interpreted.

- **Different training settings:** the two models used different epoch limits, patience and learning-rate schedules, so their training times are not directly comparable.
- **Hardware-specific timings:** Colab hardware can differ between sessions; the timings cannot be generalised to all systems.
- **Labelling choice:** bundle-branch-block beats (L and R) are labelled Normal by the team's definition, and the task is binary rather than multi-class.
- **Input assumptions:** the models were trained on MIT-BIH MLII beats at 360 Hz; other leads, sampling rates or filters give unreliable predictions.
- **Small patient pool:** only 44 records are used and no external-hospital validation, demographic subgroup analysis or robustness testing was performed.
- **Not clinically deployable:** the work does not address privacy, cybersecurity, auditability, device integration or regulatory approval.

11. Conclusion

A complete GPU-accelerated ECG beat-classification prototype was implemented using the MIT-BIH Arrhythmia Database and TensorFlow/Keras. After filtering, R-peak-centred segmentation and a leakage-free record-wise split, a 1D-CNN and a CNN-LSTM were trained to separate Normal and Abnormal heartbeats. On 15,897 test beats from 7 unseen records, the 1D-CNN performed best, with 80.66% accuracy, 0.5166 F1-score and 0.8448 ROC-AUC, while the CNN-LSTM reached 76.05% accuracy and 0.4479 F1-score. Both models remain weak on the abnormal class and overfit the training records, which shows how challenging patient-independent ECG classification is. For the GPU-computing objective, the Tesla T4 reduced CNN-LSTM five-epoch training time from 297.20 s to 33.94 s (8.76×) and inference time per beat from 0.2241 ms to 0.0302 ms (7.42×). The experiment therefore demonstrates why GPUs are widely used to accelerate deep-learning workloads in smart healthcare systems. The model remains an educational prototype and is not suitable for medical diagnosis.

12. Future Scope

- Reduce overfitting with data augmentation, stronger regularisation and patient-wise cross-validation.
- Tune the decision threshold using precision–recall analysis to improve abnormal-class recall.
- Report ROC and precision–recall curves and add probability calibration.
- Use multi-class labels (for example AAMI classes N, S, V, F, Q) instead of the binary Normal/Abnormal split.
- Add the second ECG lead or use longer context windows spanning neighbouring beats.
- Repeat the timing experiments several times and report mean and standard deviation; compare several batch sizes and throughput in beats per second.
- Explore mixed-precision training/inference and GPU memory profiling.
- Add explainability methods such as Grad-CAM to show which parts of the waveform drive a prediction.
- Evaluate on additional databases to test external generalisation.

13. Healthcare and Ethical Disclaimer

This implementation is strictly for teaching and academic experimentation. It is not a medical device, has not been clinically validated, and must not be used to diagnose, treat, or make decisions about a patient. The output is a classification result (Normal / Abnormal), not a clinical diagnosis. Medical decisions must be made by appropriately qualified healthcare professionals using validated clinical systems and procedures.

14. References

[1] Moody, G. B. and Mark, R. G. "The impact of the MIT-BIH Arrhythmia Database." IEEE Engineering in Medicine and Biology Magazine, 20(3):45–50, 2001.

[2] Goldberger, A. L. et al. "PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals." Circulation, 101(23):e215–e220, 2000.

[3] WFDB Python Package. https://github.com/MIT-LCP/wfdb-python

[4] Abadi, M. et al. "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems." 2015. https://www.tensorflow.org/

[5] Chollet, F. et al. Keras. 2015. https://keras.io/

[6] Hochreiter, S. and Schmidhuber, J. "Long Short-Term Memory." Neural Computation, 9(8):1735–1780, 1997.

[7] Kingma, D. P. and Ba, J. "Adam: A Method for Stochastic Optimization." arXiv:1412.6980, 2014.

[8] Goodfellow, I., Bengio, Y., and Courville, A. Deep Learning. MIT Press, 2016.

[9] NVIDIA. CUDA Programming Model and CUDA Toolkit Documentation. https://docs.nvidia.com/cuda/

[10] Streamlit Documentation. https://streamlit.io/

Appendix A — Selected Implementation Snippets

The complete runnable implementation (notebooks, preprocessing script, trained models and Streamlit app) is provided in the project repository. The following snippets summarise the most important components used in this report.

```
import numpy as np, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

d = np.load('ecg_processed_v1.npz')
X_train, y_train = d['X_train'], d['y_train']   # (68386, 360, 1)
X_val,   y_val   = d['X_val'],   d['y_val']     # (16399, 360, 1)
X_test,  y_test  = d['X_test'],  d['y_test']    # (15897, 360, 1)

# balanced class weights from the training set
class_weight = {0: 0.5446, 1: 6.1103}

# device selection
print(tf.config.list_physical_devices('GPU'))
main_device = '/GPU:0' if RUN_GPU else '/CPU:0'

def compute_metrics(y_true, prob, thr=0.5):
    pred = (prob >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    return dict(accuracy=accuracy_score(y_true, pred),
```

```
            precision=precision_score(y_true, pred),
            recall=recall_score(y_true, pred),
            f1=f1_score(y_true, pred),
            specificity=tn / (tn + fp),
            roc_auc=roc_auc_score(y_true, prob))
```

Appendix B — Source of Every Number

Item in this report	Source file in the repository
Dataset counts and per-record beat symbols	results/dataset_stats_per_record.csv
1D-CNN test metrics, confusion matrix, parameters	results/1d_cnn_metrics.csv, 1d_cnn_classification_report.csv
CNN-LSTM test and validation metrics	results/cnn_lstm_metrics_v1.csv
CNN-LSTM epoch history (Section 7.1)	results/cnn_lstm_history_v1.csv
CPU vs GPU timing and accuracy	results/cnn_lstm_cpu_gpu_timing_v1.csv, cnn_lstm_timing_raw_v1.csv
Seed, batch size, class weights, epochs	results/cnn_lstm_experiment_settings_v1.json
CPU, GPU, CUDA, library versions	results/hardware_info_v1.json
Preprocessing and record split	preprocessing/02_preprocessing_v1.py
All figures	graphs/*.png

Appendix C — Team Contributions

No.	Member (Roll No.)	Role	Contribution
1	Ronit Jana (24/CSE/145)	Dataset + ECG Analysis	Dataset understanding, class distribution, ECG plots
2	Ritam Jana (24/CSE/138)	Data Preprocessing	Filtering, segmentation, normalisation, record-wise split
3	Rishi Raj (24/CSE/137)	1D-CNN Model	1D-CNN design, training and evaluation
4	Rohan Adak (24/CSE/143)	CNN-LSTM + GPU Performance	CNN-LSTM, CPU/GPU benchmark, demo app, report, presentation, demo video
5	Rishabh Jha (24/CSE/136)	Integration + Evaluation	Model comparison, evaluation, Streamlit demo